Deep Learning Experiment 2 — IMDB Sentiment Analysis Full Theory

Aim

The aim of this project is to classify movie reviews as:

- Positive Review
- Negative Review

using Deep Learning techniques in TensorFlow and Keras.

This is called:

Sentiment Analysis

What is Sentiment Analysis?

Sentiment Analysis means:

> detecting emotions or opinions from text.

Example:

"This movie was amazing"
→ Positive
"This movie was terrible"
→ Negative

Type of Problem

This is:

Binary Classification

Because output has only 2 classes:

- Positive
- Negative

Overall Flow of Program

Import Libraries
↓
Load IMDB Dataset
↓
Preprocess Text Data
↓
Pad Sequences
↓
Split Dataset
↓
Create CNN + LSTM Model
↓
Train Model
↓
Evaluate Accuracy
↓
Predict Sentiment
↓
Display Graphs

---

1. Importing Libraries

Libraries used:

- TensorFlow
- NumPy
- Matplotlib

Additional modules:

- IMDB Dataset
- pad_sequences

TensorFlow

Used for:

- creating neural networks
- training deep learning models

NumPy

Used for:

- numerical operations
- arrays and matrix handling

Matplotlib

Used for:

- plotting accuracy and loss graphs

IMDB Dataset

Dataset contains movie reviews.

Each review has:

- review text
- sentiment label

Labels:

1 → Positive
0 → Negative

---

2. Loading Dataset

The IMDB dataset is loaded using:

imdb.load_data()

What Happens Here?

Reviews are converted into numbers.

Because:

> neural networks cannot understand text directly.

Words become integer tokens.

Example:

"great" → 245
"bad" → 78

---

num_words = 10000

Only top 10,000 frequent words are used.

Purpose:

- reduces memory usage
- improves efficiency
- removes rare unnecessary words

---

3. Padding Sequences

Reviews have different lengths.

Example:

Review 1 → 50 words
Review 2 → 200 words
Review 3 → 120 words

Neural networks require fixed-size input.

So:

pad_sequences()

is used.

What Padding Does

Short reviews get extra zeros added.

Example:

[4, 6, 8]
↓
[0,0,0,4,6,8]

Now all reviews become equal length.

---

maxlen = 200

Every review becomes:

200 words long

---

4. Splitting Dataset

Dataset is divided into:

- training data
- validation data
- testing data

Training Data

Used to teach the model.

Validation Data

Used during training to monitor performance.

Helps detect overfitting.

Testing Data

Used after training for final evaluation.

Acts like final exam.

---

5. Neural Network Architecture

This project uses:

Hybrid CNN + LSTM Model

Very important viva point 🔥

Why Hybrid Model?

Because:

- CNN extracts important features
- LSTM understands sequence and context

Together they improve sentiment prediction.

Model Flow

Embedding Layer
↓
Conv1D Layer
↓
MaxPooling Layer
↓
LSTM Layer
↓
Dense Layers
↓
Output Layer

---

6. Embedding Layer

Embedding(num_words, 128)

Purpose of Embedding

Converts words into dense vectors.

Instead of simple numbers:
words become meaningful mathematical representations.

Example

Words with similar meaning get similar vectors.

Example:

"good" and "excellent"

will have similar embeddings.

Embedding Dimension = 128

Each word becomes:

128-dimensional vector

---

7. CNN Layer

Conv1D(64, 5, activation='relu')

What CNN Does Here?

CNN extracts important patterns from text.

Example:

- “very good”
- “extremely bad”
- “not interesting”

CNN detects useful local features.

Filters = 64

The CNN learns:

64 different feature patterns

Kernel Size = 5

CNN checks:

5 words together at a time

Activation Function Used

ReLU Activation Function

Formula:

f(x)=max(0,x)f(x)=\max(0,x)f(x)=max(0,x)

Why ReLU?

Advantages:

- fast training
- computationally efficient
- learns complex patterns
- avoids vanishing gradient problem

---

8. MaxPooling Layer

MaxPooling1D(2)

Purpose

Reduces feature size.

Keeps most important information only.

Advantages:

- faster computation
- less memory usage
- prevents overfitting

---

9. LSTM Layer

LSTM(64)

What is LSTM?

Full form:

Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN).

Purpose of LSTM

LSTM remembers previous words and context.

Example:

"This movie is not good"

The word:

"not"

changes meaning completely.

LSTM understands this sequence relationship.

Why LSTM is Used?

Because text data is sequential.

Word order matters.

---

return_sequences=True

This means:

- first LSTM passes full sequence output to next LSTM layer.

Used when stacking multiple LSTM layers.

---

10. Dense Layers

Dense layers perform final learning and classification.

Example:

Dense(64, activation='relu')

These layers combine extracted features and make final decisions.

---

11. Dropout Layer

Dropout(0.5)

Purpose

Prevents overfitting.

During training:

- randomly disables neurons

This forces generalized learning.

---

12. Output Layer

Final layer predicts:

- Positive
  OR
- Negative

Since this is binary classification:

- one output neuron is used

Usually:

Sigmoid Activation Function

Formula:

σ(x)=11+e−x\sigma(x)=\frac{1}{1+e^{-x}}σ(x)=1+e−x1

What Sigmoid Does

Converts output into probability between:

0 and 1

Example:

0.95 → Positive
0.10 → Negative

---

13. Model Training

Training is done using:

model.fit()

Training Flow

Input Review
↓
Prediction
↓
Compare with Actual Label
↓
Calculate Error
↓
Update Weights
↓
Repeat Many Times

Epoch

One epoch means:

complete dataset passed through model once.

Example:

epochs = 10

means:
model studies full dataset 10 times.

Batch Size

batch_size = 128

Dataset is divided into small batches.

Example:

If dataset has:

1280 samples

then:

10 batches

are formed.

Iteration Formula

Iterations per Epoch=Total SamplesBatch Size\text{Iterations per Epoch}=\frac{\text{Total Samples}}{\text{Batch Size}}Iterations per Epoch=Batch SizeTotal Samples

---

Loss Function

For binary classification:

Binary Crossentropy Loss

Formula:

L=−[ylog(p)+(1−y)log(1−p)]L=-[y\log(p)+(1-y)\log(1-p)]L=−[ylog(p)+(1−y)log(1−p)]

Simple Meaning

Measures difference between:

- actual label
- predicted probability

Lower loss means better predictions.

Optimizer

The optimizer used is:

Adam Optimizer

Purpose:

- updates weights efficiently
- reduces loss
- speeds up learning

Backpropagation

Backpropagation is learning mechanism of neural network.

Steps:

1. calculate error
2. send error backward
3. update weights

4. improve future predictions

EarlyStopping Callback

Purpose:

- stops training automatically if validation loss stops improving

Advantages:

- prevents overfitting
- saves training time

ReduceLROnPlateau

If learning becomes slow:

- learning rate is automatically reduced

This helps stable convergence.

Evaluation

Model performance is checked using:

Accuracy

Example:

Accuracy = 90%

means:
90% reviews classified correctly.

Confusion Matrix

Used to compare:

- actual labels
- predicted labels

Shows:

- correct positives
- correct negatives
- wrong predictions

Prediction Function

The function:

predict_review()

takes a custom movie review and predicts:

- Positive
  OR
- Negative

with confidence score.

Final Conclusion

This project performs sentiment analysis on IMDB movie reviews using a hybrid CNN-LSTM Deep Learning model. Text reviews are converted into padded numerical sequences and passed through embedding, convolutional, pooling, and LSTM layers. ReLU activation, sigmoid output, dropout, Adam optimizer, backpropagation, and binary crossentropy loss are used during training. The trained model successfully classifies movie reviews as positive or negative with high accuracy.

explain for three
Stopped thinking

I only have the uploaded notebooks for Experiment 1 and Experiment 2 right now.
For Experiment 3, upload the `.ipynb` file (or screenshot/code), and I'll break it down exactly the same way:

- full theory
- activation functions
- epochs/batches
- architecture flow
- easy viva answers
- 20-second summary
- technical explanations in simple language

Upload the Experiment 3 notebook and I'll make it exam-ready 👍

DLL_Exp_03.ipynb
File
DLL_Exp_04.ipynb
File
exaplain for this two also
Stopped thinking

Experiment 3 — Fashion MNIST Classification using CNN

Aim

The aim of this project is to classify fashion clothing images into different categories using a:

Convolutional Neural Network (CNN)

implemented using TensorFlow and Keras.

Problem Type

This is:

Multi-Class Image Classification

because there are:

10 clothing categories

Examples:

- T-shirt
- Shoe
- Bag
- Dress
- Trouser

Overall Flow

Import Libraries
↓
Load Fashion MNIST Dataset
↓
Normalize Images
↓
Reshape Images
↓
Build CNN Model
↓
Train Model
↓
Evaluate Accuracy
↓
Predict Clothing Category
↓
Display Graphs

Dataset

Fashion MNIST contains:

70,000 grayscale images

Image size:

28 × 28 pixels

Split:

- 60,000 training images
- 10,000 testing images

Data Preprocessing

Normalization

Pixel values range:

0 to 255

They are converted to:

0 to 1

using:

X_train / 255.0

Why Normalize?

Advantages:

- faster training
- stable learning
- better accuracy

Reshaping Images

CNN expects:

(height, width, channels)

So images become:

28 × 28 × 1

1 means grayscale channel.

CNN Architecture

Model contains:

- Convolution Layers
- Batch Normalization
- MaxPooling
- Dropout
- Dense Layers

---

1. Convolution Layer

Example:

Conv2D(32, (3,3), activation='relu')

Purpose

CNN automatically detects image features like:

- edges
- shapes
- textures
- patterns

Filters = 32

Means:

32 feature detectors

are used.

Each filter learns different image patterns.

Kernel Size = (3×3)

CNN checks:

3×3 pixel region

at a time.

Activation Function

ReLU Activation

Formula:

f(x)=max(0,x)f(x)=\max(0,x)f(x)=max(0,x)

Why ReLU?

Advantages:

- fast computation
- avoids vanishing gradient
- improves training speed
- learns complex patterns

Batch Normalization

Purpose:

- stabilizes training
- improves convergence
- speeds up learning

MaxPooling Layer

Example:

MaxPooling2D((2,2))

Purpose

Reduces image dimensions.

Example:

28×28 → 14×14

Advantages:

- less computation
- faster training
- reduces overfitting

Dropout Layer

Example:

Dropout(0.25)

Purpose:

- prevents overfitting

During training:

- some neurons are randomly disabled

This forces generalized learning.

Flatten Layer

Converts 2D feature maps into 1D vector.

Needed before Dense layers.

Example:

2D image features
↓
1D vector

Dense Layers

Dense layers perform final classification learning.

Example:

Dense(128, activation='relu')

Output Layer

Since there are:

10 classes

output layer contains:

10 neurons

Usually:

Softmax Activation Function

Formula:

$$Softmax(z_i)=\frac{e^{z_i}}{\sum e^{z_j}}$$

What Softmax Does

Converts outputs into probabilities.

Example:

Shoe → 0.85
Bag → 0.03
Dress → 0.02

Highest probability becomes prediction.

Loss Function

Since this is multiclass classification:

Categorical Crossentropy

Purpose:

- measures classification error

Lower loss = better predictions.

Optimizer

Adam Optimizer

Purpose:

- updates weights efficiently
- reduces loss
- speeds up learning

Epoch

One epoch means:

> complete dataset passed through model once.

Example:

epochs = 20

means:
model studies all images 20 times.

Batch Size

batch_size = 64

Dataset is divided into smaller batches.

Advantages:

- faster training
- less memory usage
- smoother learning

Iteration Formula

Iterations per Epoch=Total SamplesBatch Size\text{Iterations per Epoch}=\frac{\text{Total Samples}}{\text{Batch Size}}Iterations per Epoch=Batch SizeTotal Samples

---

Backpropagation

Learning process of neural network.

Steps:

1. prediction
2. error calculation
3. error moves backward
4. weights updated

This gradually improves accuracy.

EarlyStopping

Stops training automatically if validation loss stops improving.

Advantages:

- prevents overfitting
- saves time

ReduceLROnPlateau

Reduces learning rate automatically when learning slows down.

Improves stable convergence.

Evaluation

Model performance measured using:

Accuracy

Example:

Accuracy = 92%

means:
92% images classified correctly.

Final Conclusion

This project uses a Convolutional Neural Network to classify Fashion MNIST clothing images into 10 categories. CNN layers extract image features, MaxPooling reduces dimensions, ReLU activation improves learning, and Softmax predicts probabilities for each class. The model is trained using Adam optimizer and categorical crossentropy loss to achieve high classification accuracy.